Desafio Final: Implementação da Watchlist (MyMovies API)

Atividade Prática Orientada por User Story | Java 21 & Spring Boot 4.1.x

> **Diretriz da Atividade**
>
> Bem-vindo ao Desafio Final da Academia Java Backend! Durante o curso, construímos a API MyMovies cobrindo REST Design, Validação, JPA/Hibernate, Integrações Externas, Observabilidade, Segurança OAuth2/JWT.
>
> Nesta atividade prática, não haverá um tutorial passo a passo de código. O seu objetivo é atuar como um Desenvolvedor Backend: analisar a User Story, planejar a sequência de tarefas necessárias e implementar a funcionalidade completa de Watchlist na API MyMovies.

1. Enquadramento e História de Utilizador (User Story)

A equipa de produto da MyMovies identificou que os utilizadores desejam guardar os seus filmes favoritos para assistir mais tarde. A funcionalidade deve ser integrada de forma **totalmente segura** na arquitetura existente.

> **User Story: US-009 - Gestão de Watchlist do Utilizador**
>
> **COMO** um utilizador autenticado na plataforma MyMovies API,
>
> **QUERO** gerir a minha lista pessoal de filmes para assistir (Watchlist), podendo consultar os meus filmes guardados, adicionar novos filmes e remover filmes da lista,
>
> **PARA QUE** possa organizar o meu consumo cinematográfico de forma personalizada, segura e individual.

Critérios de Aceitação (Acceptance Criteria):

- **Proteção de Segurança (JWT):** Todos os endpoints da Watchlist devem ser estritamente protegidos por Spring Security. Apenas pedidos com um JWT Bearer Token válido são permitidos.
- **Consulta da Watchlist:** Devolve a coleção de filmes registados na watchlist do utilizador autenticado. Caso a lista esteja vazia, devolve uma lista JSON vazia com HTTP 200 OK.
- **Adição de Filme na Watchlist:** Adiciona o filme indicado pela chave primária {movieId} à watchlist do utilizador. Deve validar se o filme existe na base de dados (caso contrário, lança 404 Not Found). Se o filme já estiver na lista do utilizador, a operação deve ser tratada adequadamente (ex.: 409 Conflict ou resposta idempotente).
- **Remoção de Filme da Watchlist:** Remove o filme {movieId} da watchlist do utilizador autenticado. Retorna HTTP 204 No Content após a remoção bem-sucedida.

2. Sugestão de Abordagens para Atacar a Demanda

Em ambiente corporativo, a equipa de engenharia pode adotar diferentes metodologias para planejar e executar a entrega de uma nova funcionalidade. Apresentamos abaixo duas abordagens válidas de mercado. Escolha a que melhor se adapta ao seu estilo de trabalho para organizar os seus passos.

Abordagem 1: Contract-First (Design-Driven / Top-Down)

Esta abordagem foca em definir primeiro as especificações de API e segurança antes de escrever qualquer lógica de persistência ou regras de negócio.

Passo	Etapa / Foco	Ações Sugeridas
Passo 1	Especificação REST & DTOs	Desenhar os contratos JSON de resposta (WatchlistResponseDTO), URIs REST (/api/movies/watchlist), métodos HTTP (GET, POST, DELETE) e criar a coleção de testes no Bruno Client com os cenários previstos.
Passo 2	Contrato de Segurança	Mapear no Spring Security quais as autorizações necessárias (ex.: escopo 'watchlist.write' ou autenticação obrigatória) e testar a rejeição de requisições sem token (401 Unauthorized).
Passo 3	Modelagem de Persistência	Modelar a estrutura de base de dados para suportar o relacionamento entre utilizador (userId) e filme (movieId), garantindo chave primária ou índice único para evitar duplicados.
Passo 4	Lógica de Serviço & Testes	Criar a camada WatchlistService integrando o Spring Data JPA para guardar, consultar e eliminar registos vinculados ao utilizador - Escrever testes unitários em JUnit e Mockito
Passo 5	Implementação Controller	Construir o WatchlistController injetando o @AuthenticationPrincipal Jwt e anotando as rotas de acordo com a especificação desenhada no Passo 1.
Passo 6	Validação End-to-End	Executar a coleção do Bruno Client gerada no Passo 1 para validar o fluxo feliz e as exceções (404 Not Found, 409 Conflict).

Abordagem 2: Domain-First / Inside-Out (Code-First & TDD)

Esta abordagem foca em construir o núcleo da aplicação (Domínio, Entidades e Regras de Negócio com Testes Unitários) e só depois expor o comportamento via Web e Segurança.

Passo	Etapa / Foco	Ações Sugeridas
Passo 1	Entidades & Repositório JPA	Definir a entidade de domínio JPA Watchlist (mapeando userId, movieId, data de adição) e criar a interface WatchlistRepository com os métodos de busca por userId.
Passo 2	Serviço & Testes Unitários	Desenvolver o WatchlistService contendo a regra de negócio e escrever testes unitários em JUnit e Mockito para garantir isolamento e validação de filmes inexistentes.
Passo 3	Mapeamento DTO & Controller	Criar os DTOs em Java 21 Records e construir o WatchlistController com os endpoints GET, POST e DELETE delegando a chamada ao serviço.
Passo 4	Integração Spring Security	Injetar a anotação @AuthenticationPrincipal Jwt no Controller e aplicar anotações de autorização.
Passo 5	Tratamento Global de Erros	Garantir que exceções lançadas no serviço (ex.: filme não encontrado) sejam convertidas em respostas HTTP adequadas via @RestControllerAdvice.
Passo 6	Testes de Integração & Bruno	Testar a aplicação de ponta a ponta utilizando o Bruno Client com múltiplos tokens JWT para validar a separação estrita de dados por utilizador.

3. Resultados Esperados

- **Código-Fonte Integrado:** O projeto Java 21 / Spring Boot 4.1.x deve compilar perfeitamente sem erros.
- **Coleção Bruno Client:** Requisições e variáveis de ambiente configuradas para a Watchlist.
- **Testes Automatizados:** Pelo menos um teste unitário de serviço ou teste de integração de controller validando o comportamento da Watchlist.
- **Arquitetura Limpa:** Segregação adequada de responsabilidades entre Controller, Service, Repository e DTOs, mantendo o código limpo e idiomático do Java 21.